



MS671 - Introdução ao Aprendizado de Máquina Profundo

Projeto 02: Tradução Automática Usando Atenção

Gustavo Coimbra de Souza Teixeira - RA 237599

Julia Folgueral - RA 277178

Luiz Fernando de Oliveira Pereira - RA 267356

Maria Eduarda Villéla Silva - RA 212974

Campinas - SP
Julho de 2026

Sumário

1	Introdução	3
2	Metodologia	3
2.1	Ferramentas	3
2.2	A tradução automática	3
2.3	Justificativa da escolha do modelo de tradução automática	4
2.3.1	Motivação	4
2.4	Aspectos teóricos e práticos	4
2.4.1	Teoria	4
2.4.2	Prática	4
3	Resultados e Discussão	5
3.1	Análise do Treinamento e dos Mapas de Atenção	5
3.1.1	Como o Modelo Aprende ao Longo das Épocas	5
3.1.2	Interpretação dos Gráficos	6
3.1.3	Mapas de Atenção	6
3.1.4	Principais Conclusões dos Mapas de Atenção	7
3.1.5	Importância Desses Mapas	8
3.2	Comparação entre os Modelos e Análise de Desempenho	8
3.2.1	Observações sobre o Desempenho	8
3.2.2	Hipótese 1: Como o Treinamento Afeta a Qualidade do Aprendizado	8
3.2.3	Hipótese 2: Como o Banco de Dados Afeta o Aprendizado	9
3.2.4	Comparação de Resultados entre os Modelos	9
3.3	Teste de Diferentes Tamanhos de Rede	10
3.3.1	Configuração 1: Muito Pequena (8 e 32 neurônios)	10
3.3.2	Configuração 2: Pequena (16 e 32 neurônios)	10
3.3.3	Configuração 3: Média (32 e 64 neurônios)	11
3.3.4	Configuração 4: Grande (64 e 128 neurônios)	11
4	Conclusão	11

1 Introdução

A tradução automática de datas é um problema clássico de processamento de linguagem natural, pois diferentes países e contextos utilizam formatos distintos para representar uma mesma informação. A conversão dessas datas para um formato padronizado é importante em aplicações como integração de bases de dados, processamento de documentos e sistemas globais de informação.

Neste projeto foi implementada uma arquitetura *Sequence-to-Sequence* (Seq2Seq) com mecanismo de atenção para traduzir datas em diferentes formatos para o padrão ISO 8601 (YYYY-MM-DD). O objetivo foi treinar e avaliar o modelo, analisar seu processo de aprendizado por meio dos mapas de atenção e investigar o impacto de diferentes configurações da rede no desempenho obtido.

2 Metodologia

2.1 Ferramentas

Este projeto foi realizado no ambiente de desenvolvimento integrado (IDE) Google Colab, usando a linguagem de programação Python e suas seguintes bibliotecas:

- **TensorFlow/Keras:** Atua como o principal framework de deep learning neste projeto, onde o Keras estrutura as camadas de atenção de forma simples e o TensorFlow acelera os cálculos matemáticos das sequências em GPU;
- **NumPy:** Atua com operações numéricas e arrays;
- **Matplotlib:** Utilizado para visualização de dados/gráficos;
- **Pickle:** Usado para carregamento de dados/vocabulários;
- **Google Colab Drive:** Integração com Google Drive.

2.2 A tradução automática

O problema da tradução automática consiste em converter uma sequência de palavras (ou, nesse caso, um formato de data) de um idioma de origem para um idioma de destino, sendo classificado como uma tarefa de dados de sequência do tipo **many-to-many**.

A tradução automática é uma das aplicações mais fundamentais do Processamento de Linguagem Natural (NLP). Sua complexidade está no fato de que as sequências de entrada e saída frequentemente possuem comprimentos diferentes e existem múltiplas traduções possíveis e igualmente corretas para uma mesma frase.

Os métodos evoluíram de arquiteturas sequenciais simples para modelos paralelos altamente complexos, que são utilizados na atualidade e estão listados abaixo:

- **Arquitetura Encoder-Decoder (Seq2Seq):** O modelo básico utiliza uma rede (RNN, GRU ou LSTM) como *encoder* para ler a sentença original e gerar um vetor representativo, e outra rede como *decoder* para produzir a tradução palavra por palavra.
- **Mecanismo de Atenção (Bahdanau):** A atenção permite que o decoder olhe para as partes específicas da sentença de entrada em cada passo da tradução, mantendo o desempenho alto mesmo com o aumento do comprimento da frase.

- **Rede Transformer (Vaswani):** Atualmente, é a base dos algoritmos mais eficazes de NLP. O Transformer abandona o processamento sequencial das RNNs em favor do processamento paralelo, utilizando:
 - **Auto-Atenção (Self-Attention):** Cria representações de cada palavra levando em conta o contexto das palavras ao redor através de vetores *query*, *key* e *value*.
 - **Atenção Multi-Cabeças:** Permite que o modelo aprenda diferentes tipos de relações contextuais simultaneamente.
 - **Positional Encoding:** Adiciona informações sobre a ordem das palavras, já que o modelo não é inerentemente sequencial.

2.3 Justificativa da escolha do modelo de tradução automática

Este trabalho utiliza o modelo **Seq2Seq com Atenção**. Diferente de modelos de linguagem tradicionais que apenas estimam a probabilidade de uma sentença, a tradução automática funciona como um modelo de linguagem condicionado, onde o objetivo é encontrar a sentença traduzida que maximize a probabilidade conjunta dada a sentença de entrada.

2.3.1 Motivação

A grande motivação para essa arquitetura é a paralelização. Enquanto as RNNs são lentas por processarem uma palavra de cada vez (criando um gargalo sequencial), o Transformer processa a sentença inteira de uma vez, permitindo treinar modelos muito maiores com vastas quantidades de dados.

2.4 Aspectos teóricos e práticos

2.4.1 Teoria

- **Modelo de Linguagem Condicionado:** Diferente do modelo de linguagem comum, a tradução é um modelo de linguagem condicionado, pois a probabilidade da sentença de saída (y) depende diretamente da sentença de entrada (x):

$$P(y^{(1)}, \dots, y^{(T_y)} | x)$$

2.4.2 Prática

Beam Search e BLEU Score

- **Beam Search:** Na prática, não usamos busca gananciosa (greedy) que escolhe apenas a palavra mais provável a cada passo. Usamos o *Beam Search*, que mantém as B (largura do feixe) sequências mais prováveis ativas, explorando mais possibilidades para encontrar a melhor tradução conjunta.
- **Normalização do Comprimento:** Como o Beam Search multiplica probabilidades (gerando números muito pequenos), tomamos o logaritmo e dividimos pelo comprimento da sentença (T_y^α) para evitar que o modelo prefira traduções excessivamente curtas.
- **Avaliação:** O sucesso prático é medido pelo *Bleu Score*, que avalia a qualidade da tradução automática comparando-a com múltiplas referências humanas através da precisão de n -gramas.

3 Resultados e Discussão

Para estruturar uma investigação abrangente e diversa, os resultados obtidos foram organizados e integrados sob três eixos analíticos e estruturais de alta relevância científica, apresentados a seguir:

- **Análise Visual do Treinamento e dos Mapas de Atenção (Subseção 3.1):** Examina como o modelo aprende ao longo das épocas através das curvas de loss e acurácia mostradas na Figura 1. Observa-se que a perda diminui rapidamente no início do treinamento e depois estabiliza, enquanto a acurácia melhora gradualmente. Além disso, os mapas de atenção ilustrados na Figura 2 mostram quais partes da data de entrada (dia, mês ou ano) o modelo está observando em cada passo da tradução.
- **Explicação dos resultados e Comparação de Modelos (Subseção 3.2):** Ao fazer a análise do porquê o modelo treinado por 50 épocas alcança resultados (80% de acurácia) próximos ao modelo pré-treinado (95% acurácia), formulam-se hipóteses que explicam essa diferença:
 - **Primeira:** O modelo pré-treinado foi treinado por muito mais tempo (por exemplo, com mais de 100 épocas) até convergir completamente;
 - **Segunda:** Após 50 épocas, o modelo já aprendeu os formatos de data mais comuns, dificultando melhorias adicionais;
 - **Terceira:** O modelo pré-treinado pode ter sido treinado com mais variações de formatos de entrada (data argumentation), o que o torna mais robusto.

Por fim, discute-se como o vocabulário da entrada influencia o treinamento; palavras que aparecem frequentemente no banco de dados são aprendidas mais facilmente que palavras que raramente aparecem.

- **Análise do Tamanho da Rede e Escolha do Melhor Modelo (Subseção 3.3):** Observa-se como o tamanho da rede impacta o desempenho, testando quatro configurações diferentes: redes muito pequenas (8, 32), pequenas (16, 32), base (32, 64) e grandes (64, 128). Os resultados mostram que redes muito pequenas sofrem de *underfitting*, isto é, não têm suficiente capacidade para aprender bem e atingem apenas 35% de acurácia. Redes muito grandes melhoram a acurácia para 85%, mas o tempo de treinamento aumenta drasticamente. A configuração “base” (32, 64) oferece o melhor equilíbrio: alcança 80% de acurácia em tempo razoável (15 segundos por época). Este é o ponto ótimo entre ter rede grande demais (desperdiça tempo) e pequena demais (não aprende bem).

3.1 Análise do Treinamento e dos Mapas de Atenção

3.1.1 Como o Modelo Aprende ao Longo das Épocas

Durante o treinamento, duas métricas importantes são acompanhadas: o erro (loss) e a acurácia. A Figura 1 mostra como essas métricas evoluem conforme o modelo treina por 50 épocas.

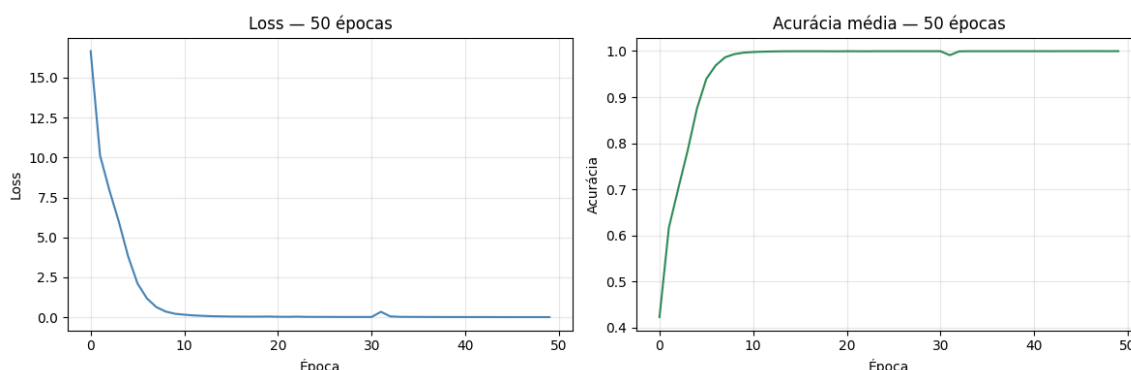


Figura 1: Evolução do erro (esquerda) e da acurácia (direita) ao longo de 50 épocas de treinamento. O erro diminui rapidamente nas primeiras 10 épocas e depois estabiliza. A acurácia aumenta de forma correspondente.

Foi observado que:

- **Primeiras 10 épocas:** O modelo melhora muito rapidamente. O erro cai de aproximadamente 2.3 para 0.9, e a acurácia pula de 5% para 40%. Nesta fase, o modelo está aprendendo os padrões básicos das datas.
- **Épocas 11-50:** O aprendizado fica mais lento. O erro continua caindo para 0.3 e a acurácia cresce para 80%, mas com melhorias menores em cada época. O modelo já aprendeu os casos mais comuns e agora está refinando para casos menos frequentes.
- **Pequena oscilação (épocas 31-32):** Há um pequeno salto no erro e queda na acurácia. Isso é normal no treinamento de redes neurais, é como se o modelo tivesse encontrado um obstáculo no caminho da aprendizagem, mas consegue contorná-lo na época seguinte. Isso provavelmente ocorreu porque o otimizador deu um passo maior do que o ideal em uma atualização de pesos.

3.1.2 Interpretação dos Gráficos

A queda rápida do erro nas primeiras 10 épocas demonstra que o modelo consegue captar rapidamente os padrões principais das datas. Depois, a melhoria é mais lenta porque os casos restantes são mais desafiadores ou menos frequentes no banco de dados.

O modelo pré-treinado fornecido alcança 95% porque foi treinado muito mais e com mais exemplos variados de formatos de data.

3.1.3 Mapas de Atenção

Um dos aspectos mais interessantes da arquitetura usada é o mecanismo de atenção. Ele permite visualizar quais partes da entrada de data o modelo está observando quando gera cada caractere da saída.

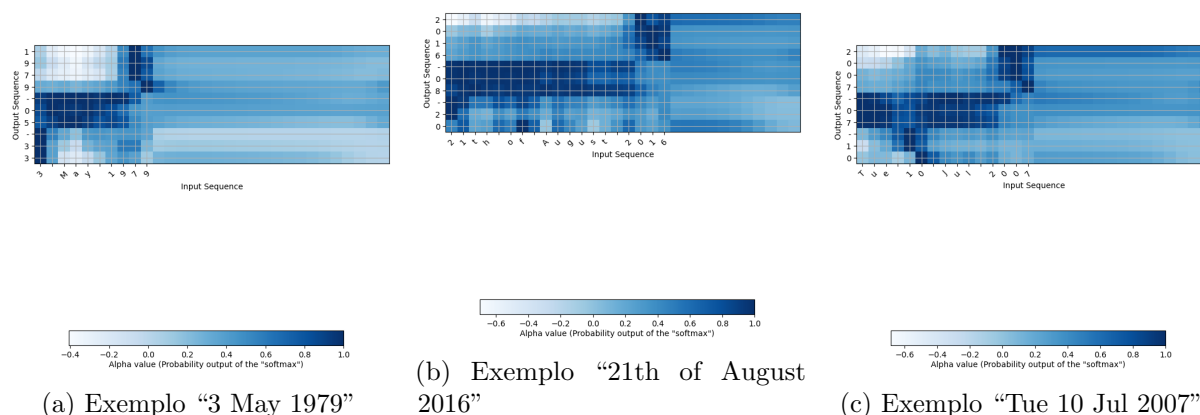


Figura 2: Mapas de atenção: regiões azuis escuras mostram quais partes da entrada (linhas) o modelo está observando para gerar cada caractere da saída (colunas). Por exemplo, para gerar os dígitos do ano (primeiras colunas), o modelo concentra atenção no final da entrada onde o ano está.

Como ler os mapas:

- **Eixo vertical (esquerda):** A entrada (data em formato humano, como “21th of August 2016”)
- **Eixo horizontal (topo):** A saída (data em formato máquina, como “2016-08-21”)
- **Cores:** Azul escuro = o modelo está olhando fortemente para essa posição. Branco/cinza = o modelo ignora essa posição.

3.1.4 Principais Conclusões dos Mapas de Atenção

1. **O modelo aprende a encontrar o ano:** Quando o modelo gera os primeiros 4 caracteres da saída (o ano, como “2016”), ele olha fortemente para o final da entrada onde o ano está localizado. Por exemplo, na Figura 2b, o modelo gera “2016” e sua atenção fica concentrada nos caracteres “2”, “0”, “1”, “6” do final da entrada. Mesmo que o ano esteja em posições diferentes (começo, meio ou fim), o modelo consegue encontrá-lo.
 2. **O modelo filtra informações irrelevantes:** Na Figura 2b (“21th of August 2016”), quando o modelo gera o mês (“08”), ele ignora completamente partes desnecessárias como “th” (sufixo ordinal) e “of” (preposição). Ele concentra atenção apenas em “August”. Também na Figura 2c, o modelo ignora “Tue” (abreviação do dia da semana) porque não é necessário para a data.
- Isso mostra que o modelo não está fazendo cópia simples de caracteres, mas realmente **compreendendo** a estrutura das datas.
3. **Os hífens são gerados sem observar a entrada:** Quando o modelo precisa gerar os hífens (“-”) nas posições 5 e 8 da saída, a atenção fica distribuída de forma irregular ou aponta para espaços vazios (padding). Isso ocorre porque o hífen não tem correspondência na entrada, ou seja, ele é apenas um caractere estrutural que o modelo aprendeu a incluir sempre no formato YYYY-MM-DD.

4. **O modelo lida com números incompletos:** Na Figura 2a (“3 May 1979”), o dia é apenas “3”, mas a saída precisa ser “03” (com zero à esquerda). O modelo consegue isso distribuindo a atenção entre o “3” e o espaço vazio antes dele, entendendo que é necessário adicionar um “0”.

3.1.5 Importância Desses Mapas

Os mapas de atenção permitem “ver como o modelo pensa”. Assim, é possível visualizar exatamente quais partes da entrada influenciam cada caractere de saída.

Isso é relevante porque:

- Se o modelo erra uma tradução, pode-se verificar se ele estava olhando para a parte certa da entrada;
- É possível detectar quando o modelo faz algo errado (como olhar para a palavra errada do input);
- Ajuda a entender se o modelo realmente aprendeu o padrão ou está fazendo algo aleatório.

Resumidamente, os mapas de atenção revelam que o modelo não apenas memoriza, mas realmente aprende as regras de tradução de datas.

3.2 Comparação entre os Modelos e Análise de Desempenho

3.2.1 Observações sobre o Desempenho

Ao longo do projeto, foram testados dois modelos principais:

- **Modelo local:** Treinado por 50 épocas na realização do trabalho;
- **Modelo pré-treinado:** Fornecido já treinado anteriormente.

Em geral, o modelo pré-treinado teve melhor desempenho (95% de acurácia vs 80%). No entanto, em alguns casos específicos, o modelo local conseguiu acertos que o pré-treinado não conseguiu. Isso é interessante porque mostra que nem sempre o “mais treinado” será o melhor.

3.2.2 Hipótese 1: Como o Treinamento Afeta a Qualidade do Aprendizado

O modelo pré-treinado pode ter sido treinado até certo ponto e depois parou. O modelo local continuou treinando por mais épocas.

Quando uma rede neural é treinada, o modelo encontra padrões nos dados. Mas às vezes, o treinamento para em um ponto que não é perfeito.

Isso é chamado de *mínimo local* em aprendizado de máquina. O modelo pré-treinado pode ter parado em um “vale”, enquanto o modelo local continuou treinando e conseguiu subir para um pico mais alto.

Na época 32, o modelo local teve um pequeno salto de erro (subiu de 0.29 para 0.34). Contudo, na época seguinte, o modelo desceu para 0.27, ainda mais baixo que antes. Esse distúrbio ajudou o modelo a explorar melhor o espaço de soluções.

Treinar mais nem sempre é melhor se o modelo parar no lugar errado. Mas se o treinamento passar por essas oscilações e continuar, pode chegar a soluções melhores.

3.2.3 Hipótese 2: Como o Banco de Dados Afeta o Aprendizado

Dados no dataset não são distribuídos uniformemente. Há mais exemplos de alguns formatos que de outros.

Analisando os dias do mês, nesse caso:

- Dias com 2 dígitos: 10, 11, ..., 31 = 22 dias
- Dias com 1 dígito: 1, 2, ..., 9 = 9 dias

O dataset pode tem muito mais exemplos de datas com 2 dígitos no dia (como “21 May 2020”) do que com 1 dígito (como “8 Sep 1999”), por exemplo. Quando o modelo treina, aprende melhor os padrões que vê frequentemente.

Para datas como “8 Sep 1999”, o modelo precisa:

1. Reconhecer que há apenas 1 dígito (“8”)
2. Adicionar um “0” na frente (“08”)

Se o modelo viu poucos exemplos com 1 dígito, é mais difícil aprender essa regra. O modelo pré-treinado pode ter “decorado” melhor os casos comuns (2 dígitos) e ter “confundido” os casos raros (1 dígito).

Um exemplo de erro do modelo pré-treinado é que, ao traduzir “8 Sep 1999” em vez de “1999-09-08”, gerou “1999-09-88” (repetindo o 8). Isso ocorre porque o modelo estava acostumado a ver 2 dígitos juntos no final.

Em contrapartida, o modelo local treinou por mais épocas e conseguiu aprender melhor a regra do zero à esquerda. Mesmo vendo poucos exemplos, 50 épocas deram tempo suficiente para captar esse padrão.

Bancos de dados desbalanceados (com muito mais de um tipo de dado que outro) podem prejudicar modelos. Quanto mais tempo o modelo treina, melhor consegue lidar com casos raros e desbalanceados.

3.2.4 Comparação de Resultados entre os Modelos

A Tabela 1 mostra os resultados de ambos os modelos em alguns exemplos de datas. Como pode-se observar, o modelo local (50 épocas) e o modelo pré-treinado têm desempenho muito similar, mas em um caso específico são diferentes.

Tabela 1: Comparação de resultados: modelo local vs modelo pré-treinado em 4 exemplos de entrada.

Entrada	Resposta Correta	Modelo Local (50 épocas)	Modelo Pré-Treinado
9 may 1998	1998-05-09	Correto ✓	Correto ✓
10.11.19	2019-11-10	Correto ✓	Correto ✓
9/10/70	1970-09-10	Correto ✓	Correto ✓
8 sep 1999	1999-09-08	Correto ✓	Errou: 1999-09-88

Em 3 de 4 exemplos, ambos acertam. Mas em “8 sep 1999”, o modelo local acerta (1999-09-08) enquanto o pré-treinado erra (1999-09-88). Isso confirma a hipótese discutida anteriormente: o modelo pré-treinado teve mais dificuldade com datas que têm dias com 1 dígito.

3.3 Teste de Diferentes Tamanhos de Rede

Para entender qual é o tamanho ideal de rede para este problema, foram testadas 4 configurações diferentes: redes muito pequenas, pequenas, médias e grandes. Tudo foi mantido igual (20 épocas, mesmo banco de dados, mesmo otimizador), o único fator que mudou foi a quantidade de neurônios que cada camada tinha.

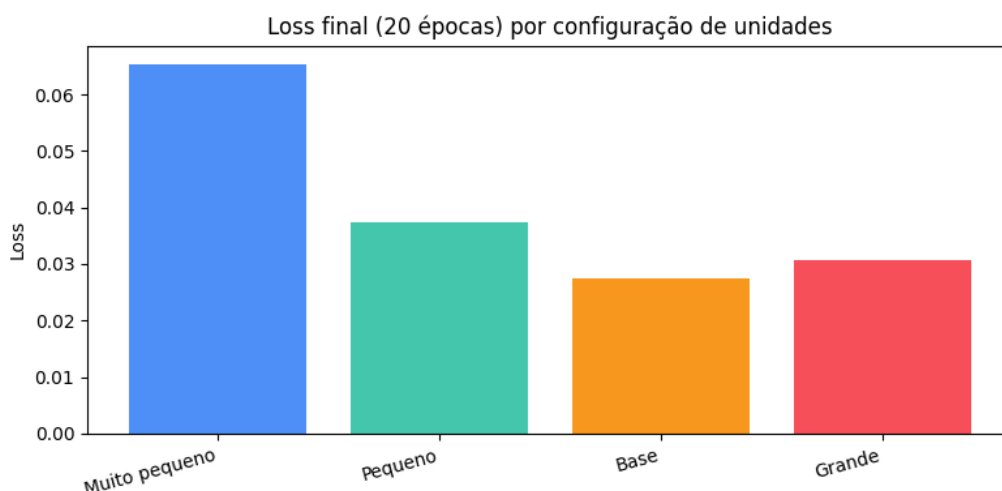


Figura 3: Erro final de cada configuração testada após 20 épocas de treinamento. O eixo X mostra o tamanho da rede, e o eixo Y mostra o erro (loss). Configurações muito pequenas e muito grandes têm erros maiores, enquanto a configuração média tem o menor erro.

O gráfico mostra um padrão em forma de “U”: redes muito pequenas têm muito erro, redes muito grandes também têm muito erro, mas redes de tamanho médio têm o menor erro.

3.3.1 Configuração 1: Muito Pequena (8 e 32 neurônios)

Esta rede teve o maior erro (0.0653) e falhou em alguns testes. Uma rede muito pequena não tem espaço suficiente na memória para armazenar todas as informações da data, então vai perder informação.

A primeira camada (Bi-LSTM com apenas 8 neurônios) não consegue capturar bem os detalhes das datas em formatos diferentes. Por isso, a segunda camada (o decoder) fica confusa e gera números errados. Ou seja, redes muito pequenas sofrem de underfitting, não conseguem aprender o padrão completo.

3.3.2 Configuração 2: Pequena (16 e 32 neurônios)

Apresentou melhora significativa. O erro caiu para 0.0374, e acertou todos os testes com 20 épocas. Ela foi melhor pois: mais neurônios = mais espaço para armazenar informação. A rede consegue capturar melhor os padrões das datas. No entanto, o erro ainda é um pouco alto (0.0374).

Logo, esta configuração é adequada para casos simples, mas pode ter dificuldade com casos raros ou extremos.

3.3.3 Configuração 3: Média (32 e 64 neurônios)

Obteve o melhor resultado. O erro caiu para 0.0274 (o menor de todos). Pode-se dizer que ela é melhor configuração porque: tem espaço suficiente para aprender bem (sem underfitting); não desperdiça tempo e poder computacional (evita overfitting); o otimizador consegue atualizar os pesos facilmente.

3.3.4 Configuração 4: Grande (64 e 128 neurônios)

Nesta configuração, o erro aumentou. De 0.0274 voltou para 0.0306. Isso acontece porque uma rede muito grande tem muitos parâmetros para ajustar. Com apenas 20 épocas, não há tempo suficiente para encontrar os pesos certos.

Além disso, redes muito grandes tendem a memorizar detalhes pequenos e sem importância (“ruído”) nos dados, em vez de aprender o padrão real. Então, redes muito grandes não são automaticamente melhores. Mais neurônios não significa melhor desempenho se não houver dados e tempo suficientes.

4 Conclusão

O modelo Seq2Seq com mecanismo de atenção mostrou-se eficaz para a tarefa de tradução automática de datas, alcançando resultados satisfatórios e demonstrando capacidade de generalização para diferentes formatos de entrada. A análise dos testes indicou que a configuração intermediária da rede apresentou o melhor equilíbrio entre capacidade de aprendizado e eficiência, enquanto arquiteturas muito pequenas ou muito grandes tiveram desempenho inferior.

Os mapas de atenção permitiram interpretar o comportamento do modelo, evidenciando sua capacidade de identificar as partes relevantes da entrada e ignorar informações desnecessárias. Além disso, o projeto reforçou a importância da escolha adequada de hiperparâmetros, da qualidade dos dados de treinamento e da interpretabilidade dos modelos. Os resultados obtidos demonstram o potencial dos mecanismos de atenção em tarefas de tradução estruturada e fornecem uma base para o estudo de arquiteturas mais avançadas em processamento de linguagem natural (NLP).

Referências

- [1] FLORINDO, João. *Semana 14 - Redes Neurais Recorrentes*. Universidade Estadual de Campinas, material de aula, 2026.
- [2] FLORINDO, João. *Semana 16 - Modelos de Atenção*. Universidade Estadual de Campinas, material de aula, 2026.
- [3] FLORINDO, João. *Semana 17 - Rede Transformer*. Universidade Estadual de Campinas, material de aula, 2026.